Portfolio

Objektorientierte und funktionale Programmierung mit Python
DLBDSOOFPP01_D

IU Internationale Hochschule

Studiengang: Cyber Security

Dashboard für das Studium – Reflexions- und Entwurfsdokument

Marius Burkhardt

Matrikelnummer: IU14107468

Tutor: Aiko Walte

Abgabedatum: 09.11.2026

Inhalt

Abbildungsverzeichnis II

1 Ziel und Vorgehen der Phase 2 1

1.1 Funktionalitäten 1

1.2 Modellierungsentscheidungen/Änderungen Phase 1 1

2 OOP-Konzepte und Python-Umsetzung 1

2.1 Dataclasses 1

2.2 Validierung 2

2.3 Enums 2

2.4 Festgestellte Besonderheiten in Bezug auf Python 2

2.5 Vererbung 2

2.6 Beziehungen 3

2.7 Testprogramme 4

2.8 Mein persönliches Fazit 5

3 Gesamtarchitektur - Schichten und Datenflüsse 5

Abbildungsverzeichnis

Abbildung 1: Tabelle Schichten 5

Abbildung 2: Gesamtübersicht UML 7

1 Ziel und Vorgehen der Phase 2

Das Ziel soll ein Python-Prototyp zur Übersicht des Studienfortschritts im Studiengang Cyber Security sein. Die Architektur orientiert sich am erarbeiteten Phase-1-Modell und den Grundsätzen der Objektorientierten Programmierung, sowie dem generellen Schichtenmodell. Hierbei sollen Verantwortlichkeiten sauber getrennt sein, sowie die Architektur definiert und erklärt werden. Weiterhin wird das UML-Diagramm ergänzt und vervollständigt.

1.1 Funktionalitäten

Zum ersten Start soll eine leere Datenbank angelegt werden. Solange kein Studiengang gespeichert ist, wird in den entsprechenden Feldern die Meldung „Kerndaten anpassen“ gezeigt. Hier soll es zwei Möglichkeiten geben: Kerndaten initial anpassen oder einen gespeicherten Datensatz im JSON-Format einlesen. Ein Datenimport ersetzt die gespeicherten Daten direkt. Eine entsprechende Beispieldatei wird mitgeliefert (examples/beispielstudiengang.json). Das Dashboard soll im oberen Teil die Kerndaten und im linken Teil die Statuswerte enthalten. Funktionale Bedienelemente sollen auf der rechten Seite des Dashboards verfügbar sein. Darin eingebettet soll eine Auflistung der Module ersichtlich sein. Es soll die Möglichkeit geben, Module hinzuzufügen oder zu ändern, die Kerndaten anzupassen und die in der Datenbank gespeicherten Daten, also den kompletten Studiengang, zu importieren oder exportieren.

1.2 Modellierungsentscheidungen/Änderungen Phase 1

Das Phase-1-Klassenmodell beschreibt Fachattribute und Beziehungen. Die Validierung konkreter Eingaben wurde als Umsetzungsentscheidung in Phase 2 ergänzt. Weiterhin wurde für Tests die Bibliothek *pytest* verwendet. Viele Attribute wurden im Fachmodell in Phase 2 als öffentlich gekennzeichnet, und nicht wie in Phase 1 als privat.

2 OOP-Konzepte und Python-Umsetzung

2.1 Dataclasses

Die Verwendung von @dataclasses vor der Definition der Klassen *Modul, Pruefungsleistung, Semester und Studiengang* nimmt viel manuelle Arbeit ab, da sinnvolle Methoden automatisch erzeugt werden. Somit können mehrere kleine Klassen innerhalb kurzer Zeit mit vergleichbar wenig Code erstellt werden, da Methoden wie beispielsweise __*init*__, oder __eq__ bereitgestellt werden.

2.2 Validierung

Die Notwendigkeit einer Validierung wurde bei ersten Tests schnell bewusst, dass die Basis einer guten Auswertung immer ein gute Datenbasis bedingt. Somit sollte unbedingt geprüft werden, dass nur gültige Werte eingegeben werden können. Um Benutzerfreundlichkeit zu steigern, sollen entsprechende Fehlermeldungen generiert werden:

```
def __post_init__(self) -> None:
        """Prüft Modulnamen und ECTS-Wert."""
        if not self.name.strip():
            raise ValueError("Der Modulname darf nicht leer sein.")
        if type(self.ects) is not int or self.ects <= 0:
            raise ValueError(
                "Die ECTS-Punktzahl muss eine positive ganze Zahl sein."
            )
```

Die Klasse Module lehnt leere Namen sowie nicht positive oder nicht ganzzahlige ECTS ab.

2.3 Enums

Innerhalb der Klasse *Pruefungsleistung* werden für den *Prüfungstyp* und den *Status* Enums definiert. Somit besteht nur eine fest definierte Anzahl an Auswahlmöglichkeiten, was Schreibfehler und Schreibarbeit minimiert, und eine eventuell später implementierte Filterung erleichtert. Fazit: In diesem Kontext wird bei der Eingabe eine Auswahl erzwungen und diese beschränkt. Folglich werden unvollständige Datensätze vermieden und eine Validierung fällt leichter, da nicht mit Zeichenketten gearbeitet werden muss

2.4 Festgestellte Besonderheiten in Bezug auf Python

- @Dataclass erzeugt nützliche Standardmethoden aber ersetzt nicht die Definition der fachlichen Regeln oder die Validierung von Benutzereingaben eines Modells.
- Typannotationen helfen bei Lesbarkeit und statischer Prüfung, ersetzen aber keine Laufzeitvalidierung.
- Im Fachmodell aus Phase 1 hatte ich Attribute als privat gekennzeichnet, während sie in der Phase 2 als öffentlich dargestellt sind. Im Prototyp sind die Dataclass-Felder öffentlich und direkt zugreifbar, um die Datenmodelle einfach zugänglich zu halten. Damit ist die Kapselung bewusst begrenzt. Sollen fachliche Regeln auch bei späteren Änderungen garantiert werden, müssten Änderungen über Methoden oder Properties laufen oder die Objekte nach der Erstellung unveränderlich sein.

2.5 Vererbung

An einem hypothetischen Beispiel möchte ich zeigen, warum ich für die Prüfungsarten trotz grundsätzlich möglicher Vererbung keine Unterklassen verwendet habe.

```python
from src.domain import Assessment, AssessmentType
class ExamAssessment(Assessment):
    """Experimentelle Prüfungsart mit spezialisiertem Verhalten."""
    def description(self) -> str:
        """Liefert die Beschreibung der Prüfungsart."""
        return "Klausur"
class PortfolioAssessment(Assessment):
    """Experimentelle Prüfungsart mit spezialisiertem Verhalten."""
    def description(self) -> str:
        """Liefert die Beschreibung der Prüfungsart."""
        return "Portfolio"

def test_assessment_subclasses_inherit_and_specialize_behavior() -> None:
    """Zeigt Vererbung und Methodenüberschreibung für Prüfungsarten."""
    assessments = (
        ExamAssessment(AssessmentType.EXAM),
        PortfolioAssessment(AssessmentType.PORTFOLIO),
    )
    assert all(isinstance(assessment, Assessment) for assessment in assessments)
    assert [assessment.description() for assessment in assessments] == [
        "Klausur",
        "Portfolio",
    ]
```

Das Beispiel zeigt, dass Unterklassen grundsätzlich funktionieren. Allerdings unterscheiden sich die Prüfungsarten nur in ihrer Bezeichnung. Die Eigenschaften Status, Note und Validierung werden jedoch für alle gleich behandelt. Unterklassen wie *Portfolio, Klausur* oder *Workbook* würden an dieser Stelle nur noch mehr Klassen erzeugen ohne eigenes Verhalten abzubilden. Die Prüfungsarten werden deshalb als *Enum* modelliert.

2.6 Beziehungen

StudyProgram enthält Semester-Objekte, und jedes Semester enthält Modul-Objekte. Im Fachmodell ist diese Beziehung als Komposition gedacht: Semester gehören zu einem Studiengang und Module zu einem Semester. Die Python-Listen bilden diese enthaltenen Objekte ab, erzwingen aber keine exklusive Zugehörigkeit oder einen gemeinsamen Lebenszyklus, was bedeutet, dass dieselbe Objektinstanz mehrfach referenziert werden könnte. In SQLite verweisen die gespeicherten Semester und Module über Fremdschlüssel jeweils auf ihren übergeordneten Datensatz. Durch Fremdschlüsselprüfung werden abhängige Datensätze mit ihrem übergeordneten Datensatz gelöscht. Das soll den Löschlebenszyklus in der Datenbank sichern, erzwingt aber keine exklusive Eigentümerschaft der Python-Objekte. Die Kompositionsrauten im UML beschreiben daher die fachliche Absicht, werden durch die Listenstruktur allein jedoch nicht garantiert.

2.7 Testprogramme

Die folgenden pytest-Beispiele prüfen konkrete Modellregeln und machen meine Entwurfsentscheidungen nachvollziehbar:

- Validierung: test_domain.py prüft ungültige Fachwerte; die Konstruktion endet erwartungsgemäß mit ValueError. Code-Ausschnitt:

```
def test_assessment_rejects_invalid_grade_state(
    status: AssessmentStatus, grade: float | None
) -> None:
    """Lehnt nicht passende Status- und Notenkombinationen ab."""
    with pytest.raises(ValueError):
        Assessment(AssessmentType.EXAM, status, grade)
```

- Validierung beim JSON-Import: test_transfer.py prüft, dass gebrochene und boolesche Werte abgelehnt werden:

```
@pytest.mark.parametrize(
    ("old_value", "new_value"),
    [
        ('"ects": 5', '"ects": 5.5'),
        ('"ects": 5', '"ects": true'),
        ('"required_ects": 180', '"required_ects": 180.5'),
        ('"required_ects": 180', '"required_ects": true'),
    ],
)
def test_import_rejects_fractional_or_boolean_ects(
    tmp_path: Path, old_value: str, new_value: str
) -> None:
    """Lehnt gebrochene und boolesche ECTS-Werte im JSON-Import ab."""
    path = tmp_path / "invalid_ects.json"
    export_study_program(make_program(), path)
    path.write_text(
        path.read_text(encoding="utf-8").replace(old_value, new_value, 1),
        encoding="utf-8",
    )

    with pytest.raises(ValueError):
        import_study_program(path)
```

- Beziehungen: test_transfer.py importiert verschachtelte Semester-, Modul- und Assessment-Objekte und prüft, ob die Struktur erhalten bleibt:

```
def test_export_and_import_preserve_the_study_program(
    tmp_path: Path,
) -> None:
    """Erhält Kerndaten, Semester, Module und Prüfungsnoten im JSON-Transfer."""
    path = tmp_path / "backup.json"
    program = make_program()
```

```
    export_study_program(program, path)
    assert import_study_program(path) == program
```

Der Test bestätigt den Erhalt der verschachtelten Struktur, weist jedoch nicht nach, dass die enthaltenen Objekte ausschließlich einem übergeordneten Objekt gehören.

2.8 Mein persönliches Fazit

Auf den ersten Blick scheint für den Zweck der Anwendung die Struktur sehr umfangreich. Bei näherer Betrachtung jedoch wird deutlich, dass der Programmcode nicht nur den Zweck der Funktionalität erfüllen sollte, sondern unter anderem auch nach Jahren verständlich, erweiterbar und wartbar sein. Gleichzeitig sind bestimmte Code-Passagen flexibler anpassbar da es den Austausch von kompletten Modulen möglich macht. Durch eine saubere Programmierung wird deutlich mehr Code produziert, als die reine Funktionalität verlangen würde, lässt sich so der Überblick behalten, während Programmteile unabhängig voneinander parallel entwickelt, geplant und getestet werden können. Jedoch ist die Programmierung mit Standardmodulen aufwändiger, was mir am Beispiel mit Tkinter bewusst wurde.

3 Gesamtarchitektur - Schichten und Datenflüsse

Schichtname	Klasse/Modul	Zuständigkeit
Domainklassen	src/domain/	Je eine Datei für Studiengang, Semester, Modul und Prüfungsleistung
Service	dashboard.py	ECTS-, Noten-, Zeit- und Zielberechnungen
Repository	repositories.py storage.py	Laden und Speichern eines Studiengangs; SQLite-Schema und persistente Speicherung
DTO	dto.py	Typisierte, für die Anzeige vorbereitete Daten
Controller	controller.py	Verbindet Repository und Service
Datentransfer	transfer.py	Validierter Import und Export von JSON-Sicherungen
View	ui.py	Darstellung und Benutzereingaben in Tkinter
Application	application.py main.py	Erzeugt und verbindet die Schichten; Programmeinstieg

Abbildung 1: Tabelle Schichten

Das Diagramm zeigt die technische Gesamtarchitektur der Umsetzung. Hierbei wird das sprachunabhängige Modell aus Phase 1 um die Schichten Service, Repository, DTO, Controller, Datentransfer, Oberfläche und Programmeinstieg ergänzt.

In der Application-Schicht befindet sich der Programmeinstieg in main() innerhalb main.py und ruft die Startfunktion DashboardApplication.start() aus der application.py auf. Daraufhin werden die verschiedenen Schichten miteinander verbunden und gestartet. Die Application erzeugt das Repository, den DashboardService und den Controller und startet anschließend die View.

Über die Repository-Schicht innerhalb storage.py wird der Datenbankpfad und das Tabellenschema eingerichtet. Falls keine Datenbank existiert, wird sie initial angelegt mit dem dort definierten Schema. Die konkrete SQLite-Implementierung ist nur der Application-Schicht bekannt.

Der Controller lädt das Domain-Objekt StudyProgram aus dem Repository und übergibt es dem DashboardService. Dieser berechnet aus dem Studiengang und dem Bezugsdatum die Kennzahlen und erstellt ein DashboardData-Objekt. Der Controller gibt die berechneten Dashboard-Daten an die View zurück.

Die View verwendet DashboardData für Kennzahlen und Modulliste. Für Kerndaten und Formulare greift die View zusätzlich auf das Domain-Objekt zu.

Beim Bearbeiten eines Moduls werden Prüfungsstatus und Note gemeinsam im Formular erfasst. View erzeugt dabei ein neues Assessment mit den eingegebenen Werten. Prüfung, ob Status und Note zusammenpassen: Eine Note ist nur bei einer bestandenen Prüfungsleistung zulässig, und bei einer bestandenen Prüfungsleistung muss eine gültige Note angegeben sein. Dies wird in Assessment.__post_init__ geprüft. Anschließend wird das Modul als Ganzes ersetzt und der Studiengang über das Repository gespeichert. Eine eigene Methode zum nachträglichen Ändern von Status oder Note besitzt Assessment nicht.

Das DTO berechnet nichts selbst, sondern enthält die für die Anzeige aufbereiteten Ergebnisse. Der Controller verwendet das Domain-Objekt zum Laden und Speichern, der Service für die Berechnungen und die View für die Anzeige und Formulareingaben

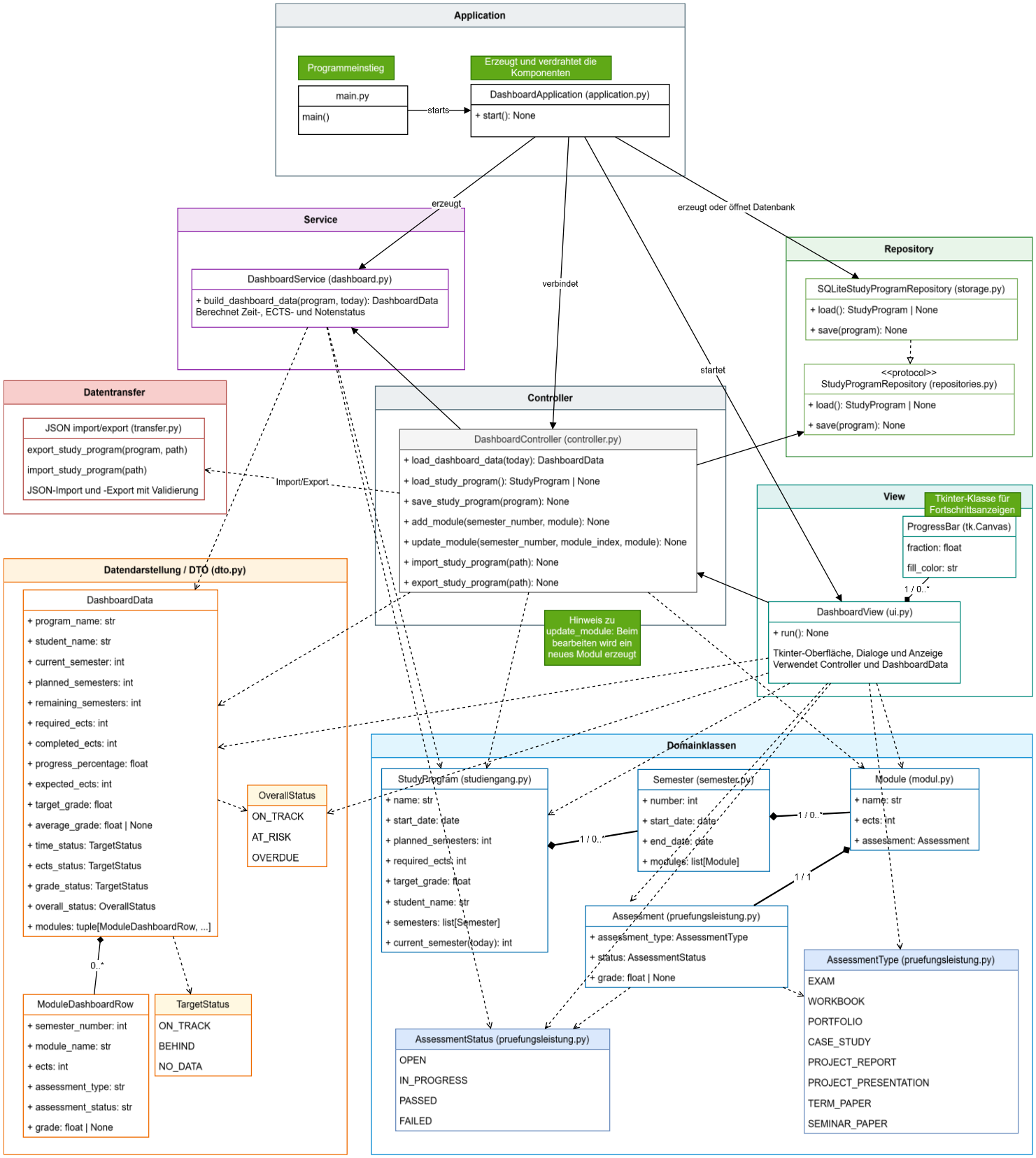


Abbildung 2: Gesamtübersicht UML